Noetia — Technical Architecture Document

Version 1.1 | May 2026

---

1. System Overview

Noetia is a multimodal reading platform composed of 9 Docker services, coordinated through a monorepo and deployed to a single VPS behind a Traefik reverse proxy. The system is designed for a small team with a low operational footprint — all services run on a single machine with minimal external dependencies.

```
                          Internet (HTTPS)
                                 │
                         Traefik v2.11
                     (SSL termination, routing)
                 ┌───────────────┴────────────────────┐
                 │                                     │
         noetia.app                           storage.noetia.app
         noetia.app/api/*                            │
                 │                              MinIO (S3)
         ┌───────┴────────┐                     (presigned URLs)
         │                │
     web:3000         api:4000
    (Next.js)         (NestJS)
                          │
           ┌──────────────┼─────────────────┐
           │              │                 │
      db:5432       cache:6379        search:7700
   (PostgreSQL)     (Redis)          (Meilisearch)
           │
     storage:9000
       (MinIO)

  Internal metrics:
    grafana:3001  prometheus:9090  (127.0.0.1 only)
```

Service Inventory

Service	Technology	Purpose
`api`	NestJS (Node.js + TypeScript)	REST API, business logic, auth, subscriptions
`web`	Next.js 14 (React)	Web reader, library, sharing, profile, admin
`db`	PostgreSQL 16	Primary data store
`cache`	Redis 7	Sessions, token store, BullMQ job queues
`storage`	MinIO (S3-compatible)	Books, audio files, generated images

Service	Technology	Purpose
`search`	Meilisearch v1.7	Full-text search for books and fragments
`worker`	BullMQ (Node.js)	Async jobs: image rendering, share exports
`image-gen`	Python 3 + Flask + Pillow	Quote card image generation microservice
`proxy`	Traefik v2.11	Reverse proxy, SSL, routing (production only)
`monitor`	Grafana + Prometheus	Metrics, alerting, dashboards

---

2. Technology Stack

Backend — NestJS API

Why NestJS: NestJS provides a structured, opinionated framework for Node.js that enforces separation of concerns through modules, controllers, services, and guards. This made it straightforward for a single backend developer to build a large API (85+ endpoints) with consistent patterns. The dependency injection system enables clean unit testing — every service can be tested with mocked dependencies.

Key architectural patterns:

- **Module-per-domain:** Each product area (auth, books, clubs, subscriptions, stats, etc.) is a self-contained NestJS module
- **JwtAuthGuard on all protected routes:** Applied at the controller level, not per-endpoint
- **ThrottlerModule:** Global 120 req/min rate limiting with per-route overrides for auth endpoints
- **TypeORM with migrations:** Zero manual SQL migrations; every schema change goes through a versioned migration file
- **Raw SQL for performance-critical queries:** Aggregate stats, streak calculations, and conflict-resolution UPSERTs use `repo.query()` directly

Authentication:

- Local (email + password) with bcrypt hashing
- JWT access tokens (15-minute expiry) + refresh tokens (stored in Redis, 7-day TTL)
- Google OAuth, Facebook OAuth, Apple Sign-In — all normalized to the same User entity
- Email confirmation tokens and password reset tokens — HMAC-signed, short-lived, stored in Redis

---

Frontend — Next.js Web

Why Next.js: Server-side rendering for SEO on public pages (landing, pricing, library discovery), client-side hydration for the interactive reader. The App Router (Next.js 14) enables route-level code splitting, which keeps the reader page bundle below 120 kB First Load JS.

Architecture decisions:

- **`'use client'` components only where needed** — data-fetching pages use server components where possible

- **`apiFetch` utility:** A thin wrapper around `fetch` that injects the JWT from localStorage and throws on non-2xx responses. Keeps all API calls consistent.
- **Custom i18n (no external library):** A `LanguageProvider` context with `useTranslation()` hook, typed translation objects for Spanish and English, and API sync on mount. Chosen over next-i18next to avoid build complexity.
- **No global state manager:** React context for auth and language; local `useState` for page-level state. The app is not complex enough to warrant Redux or Zustand.

Mobile — React Native (Expo SDK 51)

Why Expo (managed workflow): The managed Expo workflow means the `android/` and `ios/` native directories are not committed — EAS Build regenerates them via `expo prebuild` at build time. This dramatically reduces native maintenance overhead for a single frontend developer managing both platforms.

Key mobile architecture:

- **`expo-av`** for audio playback with background mode (UIBackgroundModes: audio)
- **`expo-notifications`** for push (APNs on iOS, FCM on Android via google-services.json)
- **`expo-apple-authentication`** for native Sign in with Apple
- **AsyncStorage** for offline data (downloaded book phrases, reading progress)
- **NetInfo** for offline detection and sync-on-reconnect
- **EAS Update** for OTA JavaScript updates between store builds (channel: production)

Database — PostgreSQL 16

Schema design principles:

- All primary keys are UUIDs (`uuid_generate_v4()`)
- All foreign keys have `ON DELETE CASCADE` unless data must be preserved after parent deletion
- Every multi-tenant query filters by `userId` first — no cross-user data leakage possible
- Timestamps: `createdAt` (CreateDateColumn) and `updatedAt` (UpdateDateColumn) on all core entities
- Soft deletes via `deletedAt` on club messages (content moderation requirement)

Performance indexes (migration 032):

- `idx_books_published_free` — covers the library discovery query
- `idx_books_collection` — covers collection grouping queries
- `idx_books_category` — covers category filter queries
- `idx_subscriptions_plan` — covers plan lookup on webhook processing
- `IDX_reading_stats_user_date` — covers the 7-day heartbeat UPSERT

Key tables:

Table	Row estimate (Y1)	Notes
users	5,000	Core identity, privacy settings, goals, uiLanguage
books	200	38+ Spanish public-domain + English public-domain (in dev) + growing author catalog
sync_maps	150	~1 per book; phrase array stored as JSONB
reading_progress	15,000	1 row per (user, book)
reading_stats	365,000	1 row per (user, date) — UPSERT daily
fragments	50,000	User highlights
subscriptions	5,000	1 per user
token_ledger	15,000	FIFO token redemption log
clubs	500	Public + private clubs
club_members	5,000	Many-to-many users ↔ clubs
club_messages	100,000	General club chat
club_discussions	50,000	Phrase-anchored comments

---

3. Infrastructure & Deployment

Production Server

Spec	Value
Provider	Contabo Cloud VPS 30 SSD
CPU	8 vCPU
RAM	24 GB
Storage	400 GB SSD
Network	600 Mbit/s, unlimited
OS	Ubuntu 24.04 LTS
Location	EU (Germany)

Why Contabo: Cost-optimal for the bootstrapped phase. 8 vCPU and 24 GB RAM supports 9 Docker services comfortably, with room to scale to ~10,000 active users before a VPS upgrade is needed. At that point, migrating to managed PostgreSQL (RDS/Supabase) and a larger VPS is the natural path.

Docker Compose Strategy

Three compose files separate concerns:

File	Used For
`docker-compose.yml`	Local development (volume mounts for hot reload, Mailhog for email)
`docker-compose.prod.yml`	Resource limits overlay for production
`docker-compose.server.yml`	Production deployment (Traefik labels, no nginx, no Mailhog)

Volume mount strategy (dev only): Source directories (`api/src`, `web/app`, `web/components`, `web/lib`) are mounted read-only. Changes take effect without rebuilding containers. Only changes to `package.json`, `Dockerfile`, `tsconfig.json`, or new dependencies require a rebuild.

Networking

All services join the internal `noetia` Docker network. Only Traefik has ports 80/443 exposed to the host. PostgreSQL, Redis, Meilisearch, and Grafana have no host port bindings — accessible only from within the Docker network or via SSH tunnel.

MinIO has its API accessible at `storage.noetia.app` (Traefik routes it) for presigned URL downloads. The MinIO console (port 9001) is exposed only on localhost, accessible via SSH tunnel.

---

4. API Design

Conventions

- **REST with resource-based URLs:** `GET /books/:id`, `POST /clubs`, `PATCH /users/me`
- **JWT in Authorization header:** `Bearer <token>` on all protected endpoints
- **Validation via class-validator DTOs:** All incoming request bodies are validated through NestJS pipes before reaching service logic
- **HTTP status codes strictly followed:** 201 for created resources, 204 for deletions, 400 for validation errors, 401 for auth failures, 403 for authorization failures, 404 for missing resources

Key Endpoint Groups

Group	Base Path	Notable Endpoints
Auth	`/auth`	POST register, login, logout, refresh, resend-confirmation, forgot-password, reset-password
Users	`/users`	GET/PATCH /me, DELETE /me (with ELIMINAR confirmation)
Books	`/books`	GET library, GET :id, POST progress, GET :id/sync-map, GET :id/stream
Stats	`/stats`	POST heartbeat, GET me
Subscriptions	`/subscriptions`	GET me, POST checkout, POST portal, POST webhook
Clubs	`/clubs`	Full CRUD + join, leave, ban, invite, polls, sessions, discussions

Group	Base Path	Notable Endpoints
Social	`/social`	GET :platform/status, GET :platform/connect, POST :platform/publish
Sharing	`/sharing`	POST generate (image), GET :id
Library	`/library`	GET my-books, POST redeem, GET :bookId/access

Rate Limiting

```
Global: 120 requests / 60 seconds per IP
Auth endpoints: 20 requests / 60 seconds per IP
Metrics endpoint: internal only (blocked at Nginx/Traefik level)
Stripe webhooks: exempt (ThrottlerModule @SkipThrottle)
```

---

5. Security Architecture

Authentication Security

- Passwords hashed with bcrypt (cost factor 12)
- Access tokens: 15-minute TTL (short-lived to limit damage from token theft)
- Refresh tokens: stored in Redis with 7-day TTL, rotated on each use
- Email confirmation: 24-hour HMAC-signed tokens, invalidated on use
- Password reset: 1-hour HMAC-signed tokens, invalidated on use

Authorization

- `JwtAuthGuard` validates the access token on every protected route
- `SubscriptionGuard` checks book access (subscription status OR book_purchase record) before serving sync maps or content
- Club actions check membership and role (admin-only: ban, sessions; moderator+: delete messages)
- User data isolation: every DB query filters by `userId` from the JWT subject — no path-based user ID

Infrastructure Security

- UFW firewall: only ports 80, 443, 222 open
- fail2ban: monitors port 222, max 5 retries, 1-hour ban
- SSH on port 222 (changed from default 22)
- Traefik TLS 1.2+ minimum, Let's Encrypt certificates auto-renewed
- MinIO books/ and audio/ buckets: **private** (presigned URLs required)
- MinIO images/ bucket: **public read** (generated quote cards are shareable by design)
- Social OAuth tokens encrypted at rest using AES-256-CBC before storage in Redis

Content Protection

Approach: Soft DRM (account-bound access)

Noetia uses a layered, account-bound content protection model rather than hardware DRM (Widevine/FairPlay). This is a deliberate choice appropriate to the current catalog scale and author relationships:

- **Encryption at rest:** All book text, audio files, and sync maps are stored in private MinIO buckets with AES-256 server-side encryption (MinIO SSE). Files are never directly accessible via public URL.
- **Presigned URLs with short TTL:** Streaming endpoints issue 5-minute TTL URLs (range-request audio); download endpoints issue 1-hour TTL URLs. URLs cannot be shared or replayed after expiry.
- **Account binding:** Content access is tied to the authenticated user's JWT. `SubscriptionGuard` requires a valid subscription or `book_purchase` record before any presigned URL is issued. Entitlement is derived from the JWT subject — no path-based user ID.
- **Sync map protection:** Phrase-level timestamps (`sync_maps`) are gated behind `SubscriptionGuard`. These are Noetia-proprietary assets — Whisper-aligned timestamps for public-domain books are original technical work not distributed to users.
- **No hardware DRM at current scale:** Widevine/FairPlay is not implemented. This is appropriate for the public-domain catalog and early author relationships. As publisher deals are signed (see Section 8), hardware DRM integration will be evaluated per agreement.
- **Hardware DRM roadmap:** Full Widevine (Android/web) + FairPlay (iOS) is a pre-condition for major publisher agreements. Target: Q1 2027 alongside publisher pipeline activation.

---

6. Monitoring & Observability

Metrics (Prometheus + Grafana)

- API request rate and latency (p50, p95, p99) by endpoint
- HTTP error rate (4xx, 5xx) with alerting on sustained 5xx spikes
- Container restart count (alerts on ≥ 3 restarts in 15 minutes)
- PostgreSQL connection pool usage
- Redis memory usage

Alerting

- Grafana alerts via webhook → Slack `#alerts` channel
- Alert: API 5xx rate > 5% for 3 consecutive minutes
- Alert: Container restarts > 3 in 15 minutes (15-minute grace period on startup)
- Alert: Disk usage > 80%

Logging

- Application logs via Docker JSON driver, accessible via `docker compose logs -f`
- Traefik access logs: all requests logged with status code, latency, upstream
- No centralized log aggregation at current scale (cost optimization)

Backups

- **PostgreSQL:** Daily cron at 02:00 UTC, `pg_dump` to compressed file, 7-day daily retention + 4-week Sunday retention
- **MinIO:** Weekly cron at 03:00 UTC, `mc mirror` to backup bucket, 4-copy rotation

- Both backup jobs alert on failure via Grafana

---

7. Scalability Path

Current capacity (single VPS, 8 vCPU, 24 GB): ~10,000 monthly active users

Next scaling milestone (~10K–50K MAU):

1. Migrate PostgreSQL to managed RDS (Supabase or AWS RDS) — eliminates DB maintenance burden
2. Upgrade VPS to 16 vCPU / 48 GB or split api + worker onto separate machines
3. Add CDN layer in front of MinIO for image delivery (Cloudflare proxying `storage.noetia.app`)
4. Add Redis Cluster or managed Redis (Upstash) for session and queue scaling

Future architecture (~100K+ MAU):

1. Kubernetes (K3s or managed EKS/GKE) for horizontal scaling of api and web
2. Separate read replicas for PostgreSQL (analytics and stats queries)
3. Meilisearch Cloud (eliminate self-hosted search maintenance)
4. Dedicated image generation cluster (GPU-accelerated for future AI-generated covers)

---

8. Content Pipeline & Licensing Architecture

Catalog Track 1 — Public Domain

Sources: Project Gutenberg (text), LibriVox (audio CC0), Wikisource (Spanish text) **License:** Public domain / Creative Commons Zero — no licensing fees, no royalties **Ingestion pipeline:**

- `seed-ingestion.ts` — fetches text via `gutenbergId` / `wikisourceTitle` fields in `catalogue.ts`
- `seed-audio.ts` / `seed-audio-stream.ts` — downloads LibriVox M4B/MP3 chapters to MinIO `audio/` bucket
- `seed-covers.ts` — fetches cover images from Open Library CDN
- Sync maps: `seed-sync-whisper.ts` runs Whisper alignment and stores phrase timestamps in `sync_maps` with `syncSource = 'whisper'`. This data is Noetia-proprietary — independent of the underlying work's copyright.

Technical enforcement: `isFree = true` on book record; `JwtAuthGuard` applies (account required); `SubscriptionGuard` bypassed for free books.

Catalog Track 2 — Author-Direct Upload

License: Non-exclusive publishing agreement between author and Noetia **Terms:** 36% author royalty on token redemptions and per-title sales; author retains all copyright; 30-day withdrawal right **Upload flow:**

1. Author submits via `/upload` portal: `.txt/.epub/.pdf` (text), MP3/M4A (audio), `.jpg/.png` (cover), `.srt/.vtt` (optional sync file)
2. Files stored in MinIO `books/` and `audio/` buckets under `pending/` prefix
3. Editorial review 3–5 business days: quality, sync validation, metadata

4. On approval: files moved to `published/` prefix; `isPublished = true`; `syncSource` updated to `srt` or `vtt` if sync file was provided
5. Revenue: split enforced at token redemption — `token_ledger` records each event; author payout calculated from ledger at billing cycle

Technical notes:

- `hostingTier` on `users` enforces catalog limits (1 / 3 / 12 books per tier)
- `uploadedById` FK on `books` links each title to its author
- `syncSource = 'srt'` or `'vtt'` when author supplies timing file; `'auto'` (all phrases at t=0) when no sync file is uploaded

---

Catalog Track 3 — Publisher Pipeline (Roadmap)

Status: No agreements signed. Active outreach; first publisher conversations targeting Q3 2026. **Proposed terms:** 50% Noetia / 50% publisher gross revenue split; per-title minimum guarantee TBD **Technical requirements before activation:**

Requirement	Status
Hardware DRM (Widevine + FairPlay)	Not implemented — required by most major publishers
Territorial access control (`book.territories[]` vs `user.country`)	Not implemented — required for licensed titles with regional restrictions
Publisher dashboard (analytics scoped to org)	Not implemented — author portal exists; publisher-grade org scoping pending
Audit-grade monthly reconciliation export	`token_ledger` is FIFO event-sourced; reporting export layer pending

Timeline: All publisher pipeline technical requirements targeted for Q1 2027.

Licensing Enforcement Technical Layer

Rule	Enforcement Point
Subscription or purchase required for paid content	`SubscriptionGuard` on `GET /books/:id/sync-map` and `GET /books/:id/stream`
Free books accessible to authenticated users only	`JwtAuthGuard` on all book routes; `isFree = true` bypasses `SubscriptionGuard`
Token redemption records author entitlement	`token_ledger` INSERT on every redemption event
Presigned URL TTL limits offline caching	MinIO TTL: 5 min (streaming), 1 hr (download)

Rule	Enforcement Point
Author catalog limits	`hostingTier` checked in upload service before accepting new submission
Territorial rights (roadmap)	`book.territories[]` vs `user.country` in `SubscriptionGuard` — not yet implemented
Sync map as proprietary asset	`sync_maps` never exposed in any public API; requires valid JWT + active entitlement

---

Document maintained by the Backend Developer and DevOps Engineer. Last updated: May 26, 2026.